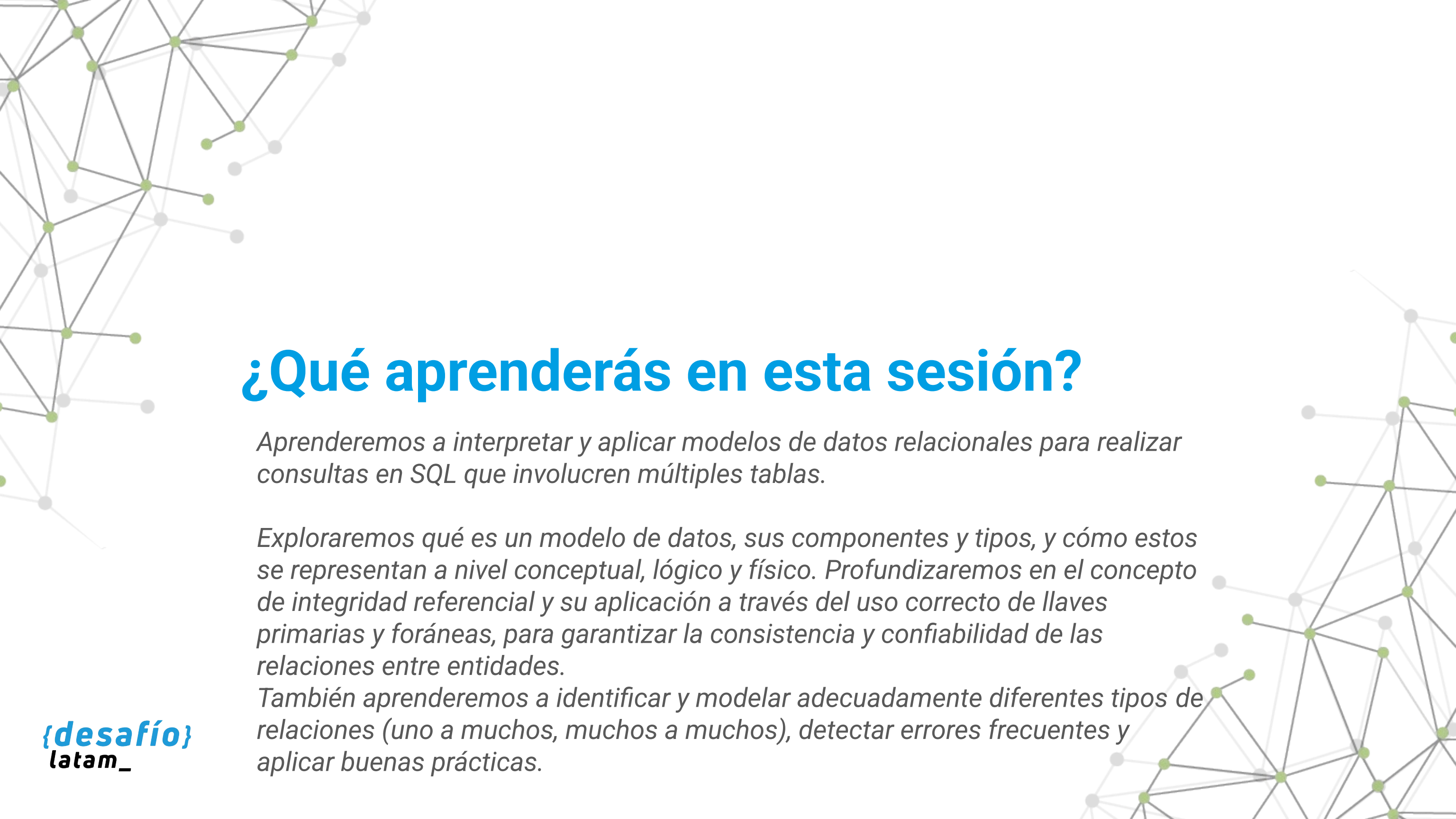




Consultas Avanzadas y Relaciones

Clase 1 - Consultas a tablas relacionadas en SQL



¿Qué aprenderás en esta sesión?

Aprenderemos a interpretar y aplicar modelos de datos relacionales para realizar consultas en SQL que involucren múltiples tablas.

Exploraremos qué es un modelo de datos, sus componentes y tipos, y cómo estos se representan a nivel conceptual, lógico y físico. Profundizaremos en el concepto de integridad referencial y su aplicación a través del uso correcto de llaves primarias y foráneas, para garantizar la consistencia y confiabilidad de las relaciones entre entidades.

También aprenderemos a identificar y modelar adecuadamente diferentes tipos de relaciones (uno a muchos, muchos a muchos), detectar errores frecuentes y aplicar buenas prácticas.



Desafío inicial

Trabajas como analista de datos para una empresa de logística que gestiona miles de órdenes de envío diarias. El equipo de ingeniería te proporciona acceso a la base de datos para que construyas un informe sobre el número de entregas por cliente, ciudad y producto. Sin embargo, al explorar la estructura, encuentras varias tablas separadas: clientes, pedidos, productos, detalle_pedido y ciudades. No hay instrucciones claras sobre cómo están relacionadas.

Tu misión

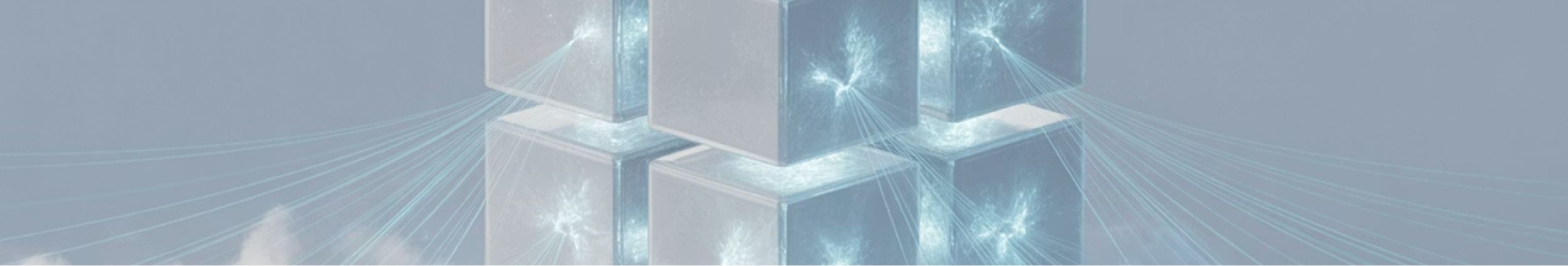
Tu misión es comprender el modelo de datos, identificar correctamente las llaves primarias y foráneas, y construir las consultas necesarias para unir estas tablas sin duplicar información ni cometer errores de integridad. ¿Por dónde empezar? ¿Cómo puedes asegurarte de que las relaciones entre las entidades estén correctamente representadas y que los datos no estén fragmentados o mal enlazados?

Para resolver este desafío, necesitas comprender profundamente qué es un modelo de datos, cómo interpretar relaciones entre tablas, y cómo implementar llaves primarias y foráneas de forma correcta.

Interpretación de modelos de datos relacionales

¿Alguna vez has necesitado cruzar información entre distintas tablas y no sabías por dónde empezar? ¿Cómo sabrías qué campos están relacionados y cuál es la lógica detrás de esa relación?

En el mundo de las bases de datos relacionales, un modelo de datos no solo representa una estructura técnica, sino que también refleja el entendimiento organizacional sobre los procesos que se quieren sistematizar. Interpretar correctamente un modelo de datos permite leer y diseñar soluciones en SQL que respeten las relaciones entre entidades y respondan eficientemente a las necesidades del negocio.



¿Qué es un modelo de datos?

Un modelo de datos es una representación abstracta que describe cómo los datos están organizados, relacionados y almacenados dentro de una base de datos. Su objetivo principal es estructurar la información de manera lógica, permitiendo mantener la integridad, reducir la redundancia y optimizar las consultas.

Los modelos de datos permiten responder preguntas como: ¿qué entidades existen?, ¿cómo se relacionan?, ¿cuáles son las reglas que rigen esas relaciones?, y ¿cómo deben consultarse los datos para extraer información útil sin comprometer su coherencia?

Componentes del modelo de datos relacional



Entidades

Representan objetos del mundo real con existencia propia, como Cliente, Producto o Pedido. Cada entidad suele corresponder a una tabla en la base de datos. Se definen por un conjunto de atributos clave que permiten identificarlas.



Relaciones

Definen cómo interactúan las entidades entre sí. Una relación puede ser uno a muchos, muchos a muchos, o uno a uno, y se expresa mediante claves foráneas en las tablas. Estas relaciones aseguran la coherencia lógica y operativa del sistema.



Atributos

Son las propiedades o características de cada entidad. Por ejemplo, la entidad Cliente puede tener los atributos nombre, ciudad, correo electrónico. Existen atributos simples y compuestos, y algunos pueden actuar como claves primarias.



Reglas de negocio

Restricciones o condiciones que los datos deben cumplir (por ejemplo, un pedido no puede existir sin un cliente). Estas reglas deben reflejar la lógica operativa y legal de una organización, y son esenciales para definir validaciones en la base de datos. Pueden expresarse como restricciones de integridad, condiciones sobre valores, o relaciones obligatorias entre entidades.

Tipos de modelos de datos

Modelo conceptual

Representa las entidades y relaciones de manera abstracta, sin detallar tipos de datos ni estructuras físicas. Se utiliza para comunicarse con usuarios no técnicos y comprender el dominio del problema. Utiliza diagramas como el Diagrama Entidad-Relación (ER).

Modelo lógico

Incorpora detalles técnicos como claves primarias, foráneas, tipos de datos, y normalización. Es la base para implementar la estructura de la base de datos. Su objetivo es traducir los requerimientos funcionales en estructuras técnicas sin estar ligado aún a un SGBD específico.

Modelo físico

Describe cómo se implementa el modelo lógico en un SGBD específico, incluyendo índices, particiones, configuraciones de almacenamiento, etc. Está condicionado por el motor de base de datos y se optimiza para el rendimiento.

Comparación entre tipos de modelos de datos

Tipo de modelo	Nivel de abstracción	Detalle técnico	Público objetivo
Conceptual	Alto	Bajo	Usuarios de negocio
Lógico	Medio	Medio-Alto	Analistas y diseñadores
Físico	Bajo	Muy alto	Administradores de BD

Tabla 1. Comparación entre tipos de modelos de datos

Fuente: Desafío Latam

Ejemplo aplicado con modelo lógico relacional

Entidad	Atributos principales	Relaciones
Cliente	id_cliente (PK), nombre, ciudad	Un cliente puede tener muchos pedidos
Pedido	id_pedido (PK), fecha, id_cliente (FK)	Cada pedido pertenece a un cliente

Tabla 2. Estructura básica de un modelo lógico relacional para clientes y pedidos

Fuente: Desafío Latam

Ejemplo aplicado (interpretación de modelo)

Si queremos consultar todos los pedidos realizados por clientes que viven en Santiago, debemos usar una consulta que relacione ambas tablas mediante la clave foránea id_cliente. Interpretar el modelo nos permite deducir que esa relación ya está establecida, y que no debemos cruzar los datos "a la fuerza" sino aprovechando la relación existente.

```
SELECT
    p.id_pedido,
    p.fecha,
    c.nombre
FROM pedidos p
JOIN clientes c ON p.id_cliente = c.id_cliente
WHERE c.ciudad = 'Santiago';
```

Buenas prácticas y errores comunes

Buenas prácticas

- Usar notación estándar para facilitar la interpretación (IE Crow's Foot, UML).
- Incluir comentarios y documentación de reglas de negocio.
- Verificar que todas las relaciones estén justificadas y normalizadas.
- Mantener consistencia en el uso de nombres y claves.

Errores comunes

- Crear relaciones sin sentido lógico solo por necesidad técnica.
- No establecer relaciones formales (por ejemplo, por falta de claves foráneas).
- Repetir información entre tablas (desnormalización innecesaria).
- No documentar reglas de negocio que expliquen por qué existe una relación.

Interpretar un modelo de datos es un paso esencial para diseñar consultas precisas, eficientes y sostenibles. Una correcta lectura del modelo permite prever qué relaciones serán necesarias en una consulta, cómo se vinculan los registros, y qué filtros son viables según la lógica del negocio. Con esta base conceptual, avanzamos hacia la comprensión de los mecanismos que garantizan la solidez del modelo: la integridad referencial.



Integridad referencial

¿Qué pasaría si eliminaras un cliente de tu base de datos sin revisar si tiene pedidos asociados? ¿Tu sistema lo permitiría? ¿Qué consecuencias traería?

En un sistema de bases de datos relacional, no basta con tener datos organizados: es fundamental que las relaciones entre ellos se mantengan coherentes. La integridad referencial es el principio que garantiza que los registros en tablas relacionadas se correspondan adecuadamente, evitando errores como pedidos sin cliente, notas sin estudiante o pagos sin factura. Su correcta implementación asegura una base sólida para las consultas complejas, el análisis de datos confiable y la automatización de procesos críticos.

Definición y principios básicos

La integridad referencial se refiere al conjunto de reglas que aseguran que una relación entre dos tablas permanezca coherente. Se implementa mediante claves primarias (PK) y claves foráneas (FK). Su principio central es:

Toda clave foránea debe tener un valor coincidente en la clave primaria de la tabla referenciada, o ser nula si la relación lo permite.

Por ejemplo, si pedidos.id_cliente es clave foránea que apunta a clientes.id_cliente, entonces no puede existir un pedido con un id_cliente que no esté registrado en la tabla clientes.

Este principio protege la consistencia de la base de datos y evita la existencia de "huérfanos" (registros sin relación válida).

Aplicación en bases relacionales

En SQL, la integridad referencial se implementa al momento de definir una clave foránea en la estructura de la tabla. Por ejemplo:

```
CREATE TABLE pedidos (  
    id_pedido INT PRIMARY KEY,  
    fecha DATE,  
    id_cliente INT,  
    FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)  
);
```

Esta sentencia crea una restricción que impide insertar en pedidos un id_cliente que no exista en la tabla clientes. Del mismo modo, impide eliminar un cliente que tenga pedidos asociados, a menos que se configure un comportamiento especial.

Estas relaciones permiten a los motores de base de datos validar automáticamente las operaciones sobre las tablas, sin necesidad de verificaciones manuales.

Cláusulas ON DELETE y ON UPDATE

Al definir claves foráneas, es posible indicar cómo debe reaccionar la base de datos cuando se actualiza o elimina un valor en la tabla referenciada. Estas cláusulas son esenciales para mantener la integridad sin bloquear operaciones necesarias.

ON DELETE CASCADE

Elimina automáticamente los registros dependientes si se elimina el registro principal.

ON DELETE SET NULL

Al eliminar el registro principal, la FK se vuelve NULL (si la columna lo permite).

ON DELETE RESTRICT

Impide eliminar si existen dependencias (valor por defecto).

ON UPDATE CASCADE

Propaga cambios del valor de la PK a las FKs relacionadas.

Ejemplo:

```
FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)
ON DELETE CASCADE
ON UPDATE CASCADE
```

Este comportamiento puede ser crítico en sistemas productivos, especialmente cuando se manejan grandes volúmenes o relaciones complejas.

Métricas clave para verificar integridad

Algunas métricas o acciones que permiten evaluar la integridad referencial de una base de datos son:

Conteo de claves huérfanas: registros con FK que no encuentran su PK asociada.

```
SELECT COUNT(*)  
FROM pedidos p  
WHERE NOT EXISTS (  
    SELECT 1  
    FROM clientes c  
    WHERE c.id_cliente = p.id_cliente  
);
```

- Porcentaje de FK válidas: indica el grado de integridad referencial en tablas con muchas relaciones.
- Tasa de errores por inserciones fallidas: cuando se intenta insertar datos con FKs inválidas.

Estas métricas permiten auditar de forma periódica el estado de integridad de los datos.

Buenas prácticas



Definir explícitamente claves foráneas al crear tablas.



Establecer políticas claras sobre ON DELETE y ON UPDATE según el modelo de negocio.



Validar integridad referencial en procesos ETL antes de cargar datos.



Documentar las relaciones y restricciones entre tablas.

Errores comunes



No declarar restricciones de clave foránea en la base de datos, confiando solo en validaciones externas.



Usar ON DELETE CASCADE sin evaluar el riesgo de eliminar grandes cantidades de datos asociados.



Cambiar manualmente valores de claves primarias sin actualizar FKs asociadas.



Permitir FKs nulas cuando la relación es obligatoria.

Comprender e implementar correctamente la integridad referencial es una de las habilidades más importantes al trabajar con bases de datos relacionales. No solo protege los datos, sino que permite construir sistemas escalables, robustos y coherentes. Con esta base asegurada, podemos avanzar al componente que habilita técnicamente estas relaciones: las claves primarias y foráneas.

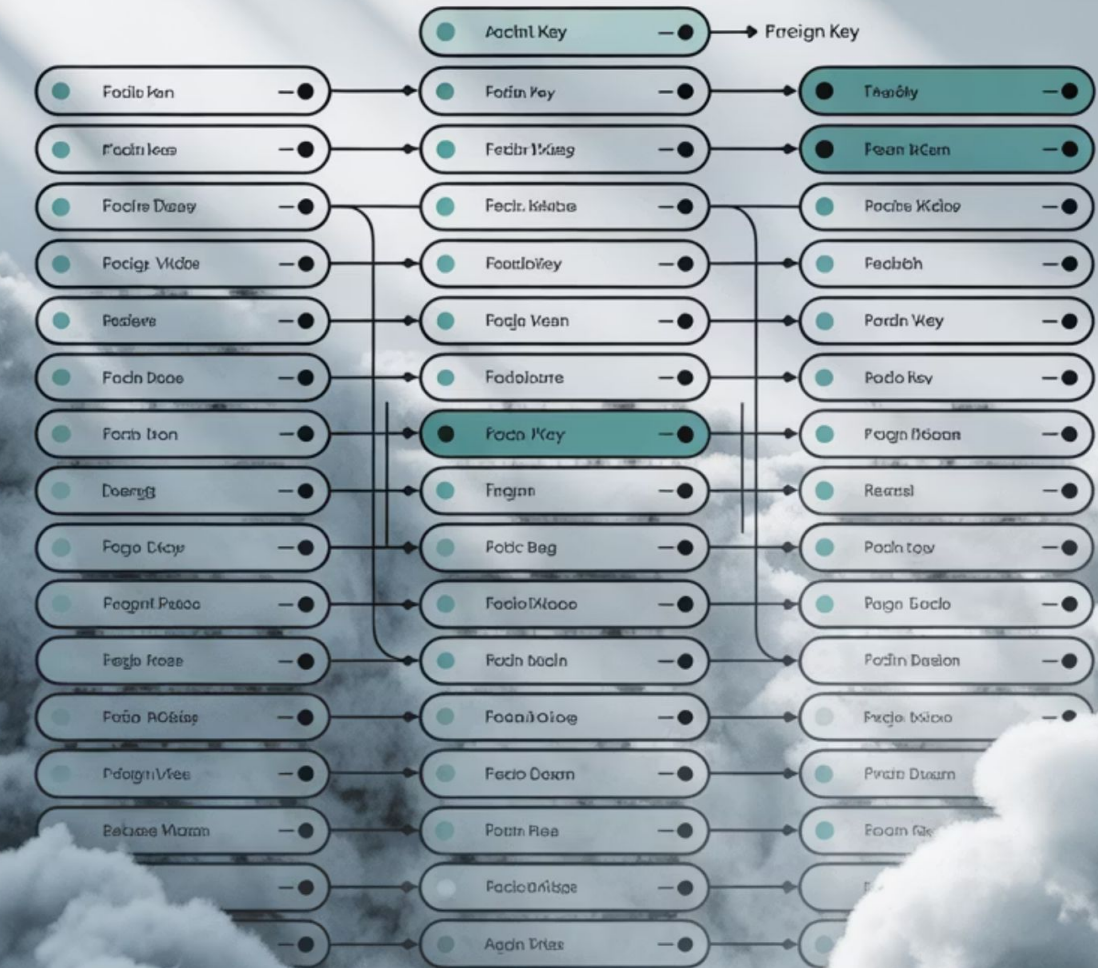
Database Schema

Llaves primarias y foráneas

¿Cómo puede una base de datos garantizar que cada cliente tenga un identificador único, y al mismo tiempo que todos los pedidos estén asociados a un cliente válido?

Las llaves primarias y foráneas son el fundamento estructural del modelo relacional.

La clave primaria permite identificar de forma única cada registro en una tabla, mientras que la clave foránea establece conexiones entre diferentes tablas, creando redes de información coherente. Entender su definición, aplicación y reglas asociadas es esencial para diseñar modelos que representen correctamente los procesos del mundo real y permitan realizar consultas eficientes y seguras.



Definición de clave primaria y clave foránea

Clave primaria (Primary Key, PK)

Es una o varias columnas que identifican de forma única cada fila en una tabla. Debe ser:

- Única (sin duplicados)
- No nula
- Inmutable (no debe cambiar frecuentemente)

```
CREATE TABLE clientes (  
  id_cliente INT PRIMARY KEY,  
  nombre VARCHAR(100),  
  correo VARCHAR(100)  
);
```

Clave foránea (Foreign Key, FK)

Es una columna que referencia la clave primaria de otra tabla, permitiendo establecer relaciones entre registros. Ejemplo:

```
CREATE TABLE pedidos (  
  id_pedido INT PRIMARY KEY,  
  fecha DATE,  
  id_cliente INT,  
  FOREIGN KEY (id_cliente) REFERENCES  
  clientes(id_cliente)  
);
```

Tipos y propiedades de llaves primarias

Tipos de llaves primarias

Llave simple: una única columna actúa como PK.

Llave compuesta: dos o más columnas combinadas actúan como PK.

Se utiliza cuando no hay un solo campo que garantice unicidad.

```
CREATE TABLE inscripciones (  
  id_estudiante INT,  
  id_curso INT,  
  fecha DATE,  
  PRIMARY KEY (id_estudiante, id_curso)  
);
```

Propiedades

Estabilidad: no debe cambiar con frecuencia.

Minimalidad: debe contener solo los atributos necesarios para garantizar unicidad.

Irreductibilidad: no debe contener atributos redundantes.

Relaciones uno a muchos, muchos a uno, y muchos a muchos



Uno a muchos (1:N)

Un registro en la tabla A se relaciona con múltiples en la B. Ej.: clientes y pedidos.



Muchos a uno (N:1)

Múltiples registros de una tabla apuntan al mismo registro en otra. Ej.: pedidos → clientes.



Muchos a muchos (N:M)

Se necesita una tabla intermedia con dos FKs. Ej.: estudiantes y cursos mediante inscripciones.

Implementación de llaves foráneas con ejemplos en SQL

Ejemplo: sistema de ventas con clientes, productos, pedidos, detalle_pedido.

```
CREATE TABLE productos (  
    id_producto INT PRIMARY KEY,  
    nombre VARCHAR(100),  
    precio DECIMAL(10,2)  
);  
  
CREATE TABLE detalle_pedido (  
    id_pedido INT,  
    id_producto INT,  
    cantidad INT,  
    PRIMARY KEY (id_pedido, id_producto),  
    FOREIGN KEY (id_pedido) REFERENCES pedidos(id_pedido),  
    FOREIGN KEY (id_producto) REFERENCES productos(id_producto)  
);
```

Esto permite que un pedido tenga múltiples productos y que un producto aparezca en muchos pedidos, manteniendo la integridad.

Validación cruzada de integridad entre claves primarias y foráneas

Comprobar si hay claves huérfanas:

```
SELECT *  
FROM detalle_pedido dp  
WHERE NOT EXISTS (  
    SELECT 1  
    FROM pedidos p  
    WHERE p.id_pedido = dp.id_pedido  
);
```

- Verificar si la FK acepta valores nulos y si esto es adecuado para el modelo.
- Implementar pruebas unitarias que verifiquen consistencia en procesos ETL.

Aplicaciones prácticas por sector económico

Sector	Ejemplo de relación PK-FK
Educación	estudiantes → inscripciones
Comercio	clientes → pedidos
Salud	pacientes → citas
Finanzas	cuentas → transacciones
Recursos Humanos	empleados → contratos

Tabla 3. Aplicaciones prácticas por sector económico

Fuente: Desafío Latam

Buenas prácticas

- Nombrar llaves claramente (ej. `pk_id_cliente`, `fk_id_cliente`).
- Documentar todas las relaciones y restricciones en el diccionario de datos.
- Utilizar índices sobre columnas FK para mejorar el rendimiento de joins.

Errores comunes

- No definir PK en tablas críticas.
- Usar datos personales (como RUT) como PK sin anonimización.
- Eliminar registros padre sin contemplar FKs hijas.
- Permitir duplicados al omitir PK o restricciones de unicidad.

Las llaves primarias y foráneas no solo permiten mantener el orden en una base de datos relacional, sino que reflejan las relaciones reales del negocio y habilitan consultas complejas sin duplicación. Una vez comprendidos estos fundamentos, estamos listos para construir consultas entre múltiples tablas, interpretar modelos de datos completos y realizar análisis relacional efectivo, que abordaremos en la siguiente sesión.

Actividad guiada:

Entendiendo relaciones en un sistema de pedidos



Objetivo de la actividad

Aplicar el análisis e interpretación de modelos de datos relacionales para identificar entidades, llaves primarias y foráneas, y construir consultas que relacionen múltiples tablas sin comprometer la integridad referencial.

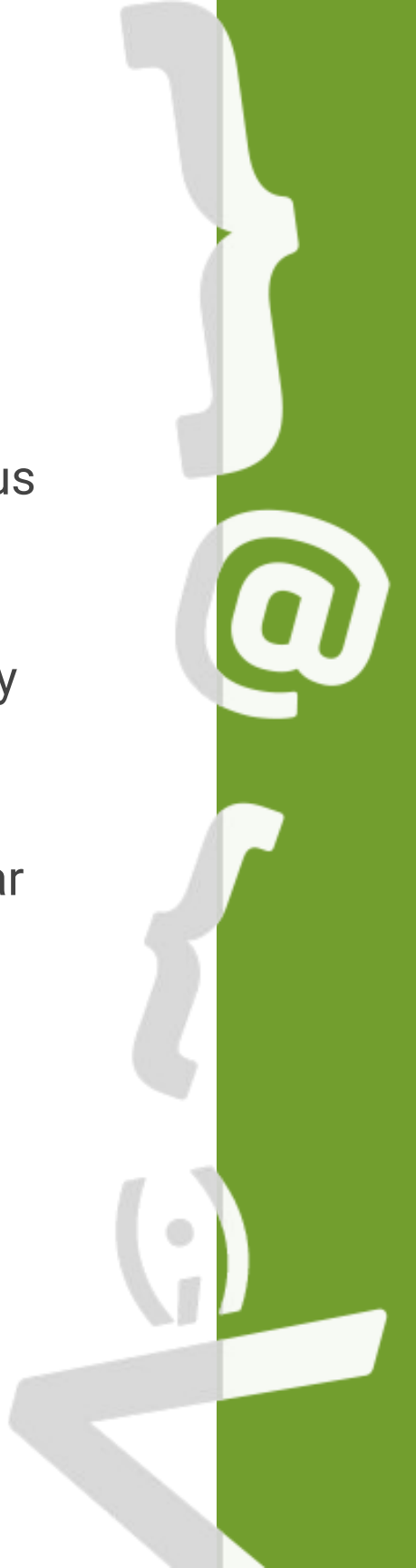


Situación problema

Formas parte del equipo de datos de una empresa de distribución de alimentos que necesita optimizar sus reportes logísticos.

Actualmente, su base de datos incluye las siguientes tablas: clientes, pedidos, detalle_pedido, productos y ciudades, pero no cuentan con un diagrama actualizado ni documentación de relaciones.

Se te encarga revisar la estructura relacional, identificar las llaves que conectan las entidades y desarrollar una consulta que relacione correctamente los datos de entregas realizadas por ciudad.



Estructura de tablas

```
-- Tabla clientes
CREATE TABLE clientes (
  id_cliente INT PRIMARY KEY,
  nombre VARCHAR(100),
  ciudad_id INT,
  FOREIGN KEY (ciudad_id) REFERENCES
ciudades(id_ciudad)
);

-- Tabla ciudades
CREATE TABLE ciudades (
  id_ciudad INT PRIMARY KEY,
  nombre_ciudad VARCHAR(100)
);

-- Tabla productos
CREATE TABLE productos (
  id_producto INT PRIMARY KEY,
  nombre_producto VARCHAR(100),
  precio DECIMAL(10,2)
);
```

```
-- Tabla pedidos
CREATE TABLE pedidos (
  id_pedido INT PRIMARY KEY,
  fecha DATE,
  id_cliente INT,
  FOREIGN KEY (id_cliente) REFERENCES
clientes(id_cliente)
);

-- Tabla detalle_pedido
CREATE TABLE detalle_pedido (
  id_detalle INT PRIMARY KEY,
  id_pedido INT,
  id_producto INT,
  cantidad INT,
  FOREIGN KEY (id_pedido) REFERENCES
pedidos(id_pedido),
  FOREIGN KEY (id_producto) REFERENCES
productos(id_producto)
);
```

Estructura de tablas

Estudia el siguiente conjunto de estructuras (simplificadas):

```
CREATE TABLE clientes (  
  id_cliente INT PRIMARY KEY,  
  nombre VARCHAR(50),  
  ciudad_id INT  
);  
  
CREATE TABLE ciudades (  
  id_ciudad INT PRIMARY KEY,  
  nombre_ciudad VARCHAR(50)  
);  
  
CREATE TABLE pedidos (  
  id_pedido INT PRIMARY KEY,  
  id_cliente INT,  
  fecha DATE,  
  FOREIGN KEY (id_cliente) REFERENCES  
clientes(id_cliente)  
);
```

```
CREATE TABLE productos (  
  id_producto INT PRIMARY KEY,  
  nombre_producto VARCHAR(50),  
  precio DECIMAL(10,2)  
);  
  
CREATE TABLE detalle_pedido (  
  id_detalle INT PRIMARY KEY,  
  id_pedido INT,  
  id_producto INT,  
  cantidad INT,  
  FOREIGN KEY (id_pedido) REFERENCES  
pedidos(id_pedido),  
  FOREIGN KEY (id_producto) REFERENCES  
productos(id_producto)  
);
```

Actividad: Interpretación del modelo relacional

Dibuja o interpreta el modelo relacional resultante de estas estructuras. Identifica:

- Entidades
- Llaves primarias
- Llaves foráneas
- Tipo de relación entre entidades (uno a muchos, muchos a muchos)

Escribe una consulta SQL que muestre, por cada cliente, su ciudad y el total de productos que ha comprado (sumando cantidades):

```
SELECT
  c.nombre AS cliente,
  ci.nombre_ciudad AS ciudad,
  SUM(dp.cantidad) AS total_productos
FROM clientes c
JOIN ciudades ci ON c.ciudad_id = ci.id_ciudad
JOIN pedidos p ON c.id_cliente = p.id_cliente
JOIN detalle_pedido dp ON p.id_pedido = dp.id_pedido
GROUP BY c.nombre, ci.nombre_ciudad;
```

Reflexión sobre integridad referencial

Reflexiona sobre: ¿Qué riesgos habría si una de las llaves foráneas no estuviera correctamente declarada? ¿Qué pasaría si un cliente fuera eliminado sin restricciones ON DELETE?

Materiales necesarios

- Entorno de desarrollo SQL (DB Fiddle, SQLiteStudio u otro).
- Código base entregado por el relator.
- Esquema del modelo relacional proyectado o impreso.



Resumen

Comprendimos qué es un modelo de datos y cómo interpretarlo en sus niveles conceptual, lógico y físico.

Analizamos los componentes clave de un modelo relacional: entidades, atributos, relaciones y reglas de negocio.

Estudiamos el concepto de integridad referencial, su relevancia para la coherencia de los datos y cómo se implementa en SQL.

Identificamos las diferencias entre llaves primarias y foráneas, su rol en la estructura relacional y cómo se utilizan para modelar relaciones entre tablas.

Aplicamos buenas prácticas para asegurar la integridad entre entidades y evitar errores comunes como duplicación, orfandad o inconsistencia.



Próxima sesión...

En la siguiente sesión exploraremos cómo realizar consultas de selección entre tablas relacionadas, profundizando en las técnicas de combinación de datos mediante INNER JOIN y LEFT JOIN.

Aprenderemos a conectar múltiples tablas de manera eficiente, respetando la integridad relacional, y a construir consultas analíticas útiles para entornos empresariales reales.

Preguntas de cierre

¿Cuál es la diferencia entre un modelo lógico y uno físico de datos?

¿Qué papel cumple la clave foránea en la integridad referencial?

¿Qué implica la cláusula ON DELETE CASCADE?

¿En qué consiste una relación muchos a muchos y cómo se implementa en SQL?

¿Cómo puedes validar que una base de datos respeta la integridad entre entidades?

Referencias bibliográficas

- Coronel, C., & Morris, S. (2022). Database Systems: Design, Implementation, & Management (14th ed.). Cengage Learning.
- Rob, P., & Coronel, C. (2020). Fundamentals of Database Systems. Cengage Learning.
- Date, C. J. (2019). An Introduction to Database Systems (8th ed.). Addison Wesley.
- PostgreSQL Global Development Group. (2024). PostgreSQL Documentation. <https://www.postgresql.org/docs/>
- Oracle. (2024). SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/>